



Facultad de Ciencias
de la **Administración**

TECNICATURA
UNIVERSITARIA EN
**DESARROLLO
WEB**



PROGRAMACIÓN I

Unidad VI – Colecciones y tipos de datos compuestos

Tuplas y Diccionarios

Tecnicatura Universitaria en Desarrollo Web

Facultad de Ciencias de la Administración

Universidad Nacional de Entre Ríos

- **Objetivos**

- Identificar características de tipos compuestos y diferencias con los elementales.
- Comprender las principales características de listas, tuplas y diccionarios y cómo hacer uso de ellas.

- **Temas a desarrollar:**

- Listas. Definición.
- Operaciones de modificación y consulta.
- Clasificación: unidimensionales, bidimensionales.
- **Tuplas. Definición. Operaciones de modificación y consulta.**
- **Diccionarios. Definición. Operaciones de modificación y consulta.**

- Al igual que las **listas**, las **tuplas** son una secuencia de valores de cualquier tipo que están indexadas por enteros. La **diferencia principal** entre una **tupla** y una **lista** es que las tuplas son **inmutables**.
- Sintácticamente una **tupla** es una lista de valores separados por comas:
 - » `t = 'a', 'b', 'c', 'd', 'e'`
- Sin embargo, aunque no es necesario, es común ver las tuplas encerradas entre paréntesis:
 - » `t = ('a', 'b', 'c', 'd', 'e')`
- Para crear una **tupla** con un único elemento, tenemos que incluir la coma al final:
 - » `t1 = 'a',`
 - » `type(t1) → <class 'tuple'>`
- Sin la coma final se trata de un string.

Tuplas (2)

- Otra forma de crear una tupla es utilizando la función **tuple()**. Sin argumentos, creamos una tupla vacía:
 - » **t = tuple()**
 - » **t → ()**
- Si pasamos por parámetro a la función **tuple()** una secuencia (string, lista o tupla), el resultado es una tupla con los elementos de la secuencia.
 - » **t = tuple('Racing')**
 - » **t → ('R', 'a', 'c', 'i', 'n', 'g')**

Tuplas - Operadores

- La mayoría de los operadores que funcionan con listas también lo hacen para las tuplas.
- El operador corchetes indexa un elemento:
 - » `t = ('a', 'b', 'c', 'd', 'e')`
 - » `t[0] → 'a'`
- Y el operador slice selecciona un rango de elementos:
 - » `t[1:3] → ('b', 'c')`
- Pero si intentamos modificar uno de los elementos de la tupla tendremos un error:
 - » `t[0] = 'A' → TypeError: object doesn't support item assignment`
- Dado que las tuplas son inmutables, no podemos modificar los elementos. Pero podemos reemplazar una tupla con otra
 - » `t = ('A', ) + t[1:]`
 - » `t → ('A', 'b', 'c', 'd', 'e')`
- Esto crea una nueva tupla y luego copia el resultado en `t`.

Asignación

- Cuando hicimos nuestros primeros ejercicios con **Python** vimos tuplas! El intercambio de valores que hacíamos con variables temporales:
 - » `temp = a`
 - » `a = b`
 - » `b = temp`
- Con **tuplas**, la solución es más elegante: `a, b = b, a`
- El **lado izquierdo** es una **tupla de variables** y el **lado derecho** es una **expresión que devuelve una tupla**. Cada valor es asignado a su respectiva variable. Todas las expresiones del lado derecho son evaluadas antes de la asignación.
- El **número de variables** del lado izquierdo y del lado derecho tienen que ser **exactamente el mismo**.
- De manera más general podemos decir que del lado derecho podemos tener cualquier tipo de secuencia (string, lista o tupla). Por ejemplo, para dividir una dirección de email entre nombre de usuario y dominio podemos escribir:
 - » `direccion = 'mfrancisconi'`
 - » `usuario, dominio = direccion.split('@')`

Tuplas y valores de retorno

- En términos estrictos **una función puede solo retornar un valor**, pero si el valor es una **tupla**, el efecto es el mismo que retornar múltiples valores.
- Por ejemplo, si queremos dividir dos enteros y computar el coeficiente y el resto, es ineficiente computar `x // y` y luego `x % y`. Es una mejor solución computar ambos al mismo tiempo.
- **Python** cuenta con la función integrada **`divmod()`** que toma dos argumentos y nos retorna una **tupla** con dos valores: el cociente y el resto.
- Podemos almacenar el resultado en una **tupla**:
 - » `t = divmod(7,3)`
 - » `t → (2, 1)`
- O utilizar la asignación de **tupla** para almacenar valores de manera separada:
 - » `cociente, resto = divmod(7, 3)`
 - » `cociente → 2`
 - » `resto → 1`

Métodos de tuplas

- Como son **inmutables**, las **tuplas** no cuentan con ningún método que permita modificar los elementos que contienen. A continuación los métodos de **tuplas**:
- **count()**: cuenta el número de veces que un valor pasado por parámetro se ha encontrado en la tupla.
- **index()**: busca el valor que se ha pasado por parámetro y devuelve el índice en el que se ha encontrado.

Diccionarios

- Los **tipos compuestos** que hemos visto hasta ahora (cadenas, listas y tuplas) usan **enteros** como **índices**.
 - Si intenta usar cualquier otro tipo como índice provocaría un error.
- *Un **diccionario** contiene una **colección de índices**, que se llaman **claves**, y una **colección de valores**.*
 - Cada **clave** está **asociada** a un **único valor**. La **asociación** entre una **clave** y un **valor** recibe el nombre de **par clave-valor**, **ítem** o **entrada**.
- Los **diccionarios** son similares a otros tipos compuestos excepto en que pueden usar como **índice cualquier tipo inmutable**.
 - Sirven para la construcción de muchos elegantes y eficientes algoritmos.
- En términos matemáticos un diccionario representa un **mapeo** desde **claves** a **valores**. De forma que para cada clave hay un valor.
 - Como ejemplo, vamos a construir un diccionario que haga un mapeo de palabras del Español al Inglés.

Diccionarios (2)

- Una forma de crear un **diccionario** es empezar con el **diccionario vacío** y añadir elementos.
 - El **diccionario vacío** se expresa usando llaves {} ó utilizando la función **dict()**.
 - » **diccionario = {}**
- Para agregar ítems en el diccionario utilizamos corchetes:
 - » **diccionario['uno'] = 'one'**
- La línea anterior crea un ítem que mapea la clave **'uno'** al valor **'one'**. Si nuevamente imprimimos el diccionario vemos el par **clave-valor** con el símbolo : (dos puntos) entre la clave y el valor.
 - » **{'uno': 'one'}**
- El formato de salida de un diccionario es también formato de entrada. Podemos crear un diccionario con tres entradas:
 - » **diccionario = {'uno': 'one', 'dos': 'two', 'tres': 'three'}**
- A la hora de imprimir por pantalla **diccionario** puede suceder que las claves no estén en el orden en que las insertamos. Esto no es un problema porque para acceder a los valores de un diccionario siempre usamos una clave como índice.
 - » **diccionario['dos'] → 'two'**

Diccionarios (2)

- Si la clave no está en el diccionario vamos a tener una excepción:

» `diccionario2['cuatro']` → `KeyError: 'cuatro'`

La función `len()` que vimos para strings, listas y tuplas tiene la misma semántica con diccionarios devolviéndonos la cantidad de pares clave-valor que hay en el diccionario.

» `len(diccionario2)` → `3`

- El operador `in` también funciona con los diccionarios, nos indica si hay alguna entrada con clave igual a la que estamos buscando.

» `'uno' in diccionario` → `True`

» `'cinco' in diccionario` → `False`

Si necesitamos controlar si hay algún valor en el diccionario podemos utilizar el método `values()` en combinación con el operador `in`:

`'one' in diccionario.values()` → `True`

`'five' in diccionario.values()` → `False`

- Utilizando `del` podemos quitar una entrada del diccionario:

» `del diccionario['uno']` → `{'dos': 'two', 'tres': 'three'}`

Diccionarios para representar entidades con varias propiedades

- En **Python** podemos utilizar los diccionarios para establecer las propiedades y valores con los que cuenta una entidad. Por ejemplo:
- » `vehículo = {
 'tipo': 'Automóvil',
 'año': 2022,
 'marca': 'Ford',
 'modelo': 'Focus ST',
 'color': 'Azul'}`
- Podemos visualizarlo también de forma tabular:



Clave	Valor
tipo	Automóvil
año	2022
marca	Ford
modelo	Focus ST
color	Azul

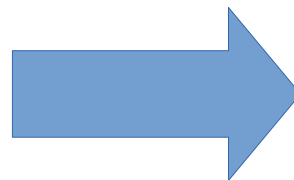
Diccionarios como colección de contadores

- Supongamos que dado un string queremos **contar** cuántas veces aparece **cada letra** en él. Hay varias soluciones posibles:
 - 1) Podemos crear **26** variables, una para cada letra del alfabeto. Luego, podemos recorrer el string y para cada carácter incrementar el contador de la variable correspondiente usando condicionales anidados.
 - 2) Una segunda alternativa es crear una lista con **26** elementos y luego convertir cada carácter a número (utilizando la función integrada **ord()**), usando el número como índice dentro de la lista y el incremento del contador apropiado.
 - 3) Podemos crear un diccionario con caracteres como claves y contadores como los valores correspondientes. La primera vez que nos encontramos con un carácter, tendremos que agregar un ítem al diccionario. Las próximas ocurrencias, bastaría con incrementar el valor de un ítem existente.
- Cada una de estas opciones, si bien lleva a cabo la misma computación, cada una de ellas la implementa de manera diferente.
- Una **implementación** es una forma de llevar a cabo una computación; algunas implementaciones son mejores que otras. Por ejemplo, la ventaja de la implementación que utiliza un **diccionario** es que **no tenemos que conocer de antemano qué letras van a aparecer en el string** y sólo guardaremos en el diccionario las letras que aparecen (ahorrando memoria).

Bucles y diccionarios

- Si usamos un diccionario en un bucle **for**, haremos un recorrido por las claves de ese diccionario. Por ejemplo, **d_print()** imprime cada clave y su correspondiente valor.

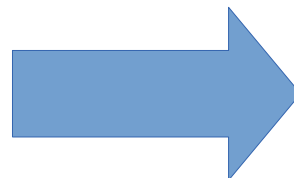
```
def d_print(dict):  
    for c in dict:  
        print(c, dict[c])  
» d_print(histograma('Lara'))
```



```
l 1  
a 2  
r 1
```

- Las claves no tienen un orden particular. Para recorrer las claves en orden tenemos que usar la función integrada **sorted()**:

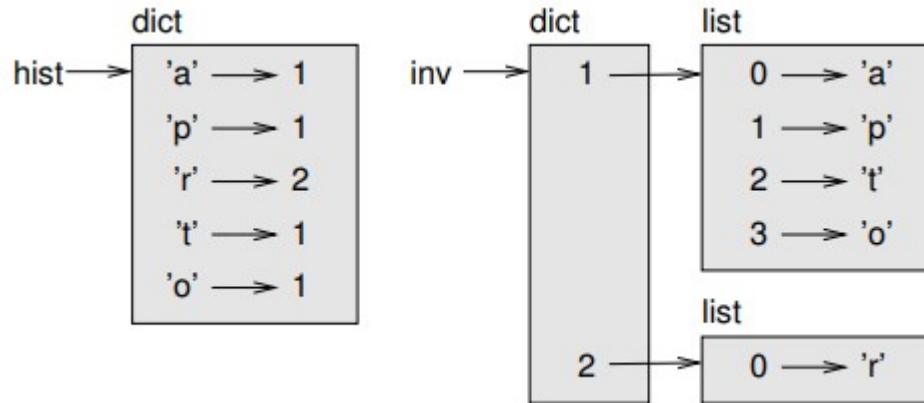
```
def d_print_sorted(dict):  
    for key in sorted(dict):  
        print(key, dict[key])  
» d_print_sorted(histograma('Lara'))
```



```
a 2  
l 1  
r 1
```

Diccionarios y listas

- Las **listas** pueden aparecer como valores de un **diccionario**.
- Dado un **diccionario** que mapea **letras** y su **frecuencia**, es posible que necesitemos invertir el diccionario. Es decir, crear un diccionario que haga un mapeo de **frecuencias** a **letras**.
 - Como podemos tener **muchas letras con la misma frecuencia**, cada valor en el diccionario invertido deberá ser una **lista de letras**.



Métodos de diccionarios

- **clear()**: elimina todo el contenido del diccionario.
- **get()**: nos permite consultar el valor para un clave determinada. El segundo parámetro es opcional, y en el caso de proporcionarlo es el valor a devolver si no se encuentra la clave.
- **items()**: devuelve una lista con las entradas del diccionario. Si se convierte en una lista se puede indexar como si de una lista normal se tratase, siendo los primeros elementos serán las claves y los segundos los valores.
- **keys()**: retorna una lista con todas las claves del diccionario.
- **values()**: devuelve una lista con todos los valores del diccionario.
- **pop()**: elimina la clave que se pasa como parámetro y devuelve su valor asociado. Daría un error si se intenta eliminar una clave que no existe.
- **popitem()**: elimina de manera aleatoria un elemento del diccionario.
- **update()**: se llama sobre un diccionario y tiene como argumento otro diccionario. Los valores son actualizados y si alguna clave del nuevo diccionario no esta, es añadida.

Bibliografía

- Óscar Ramírez Jiménez: ***“Python a fondo”*** 1era Edición. Ed. Marcombo S.L.. 2021.
- Allen Downey. ***“Think Python”***. 2Da Edición. Green Tea Press. 2015.
- Bill Lubanovic. ***“Introducing Python”***. 2Da Edición. O’ Reilly. 2020.
- Eirc Matthes: ***“Python Crash Course”***. 1era Edición. Ed. No Starch Press. 2016.
- Zed A. Shaw: ***“Learn Python 3 the Hard Way”***. 1era Edición. Ed. Addison-Wesley. 2017.